

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 2**  
**MODUL 14**  
**“PENGURUTAN PADA DATA TERSTRUKTUR”**



**DISUSUN OLEH:**  
**SALSADILLA HANNY AZIZAH**  
**108092500014**  
**S1IF-13-02**  
**DOSEN:**  
**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2024/2025**

## DASAR TEORI

Sorting atau pengurutan data merupakan suatu proses untuk menyusun data berdasarkan urutan tertentu, baik secara ascending (membesar) maupun descending (mengecil). Pengurutan data sangat penting dalam pemrograman karena dapat mempermudah proses pencarian, pengelolaan, dan pengolahan data. Pada bahasa pemrograman Go, proses sorting dapat dilakukan menggunakan berbagai algoritma, salah satunya adalah selection sort dan insertion sort.

Selection sort merupakan algoritma pengurutan yang bekerja dengan cara mencari nilai terkecil atau terbesar dari sekumpulan data, kemudian menukarnya dengan elemen pada posisi yang sesuai. Proses ini dilakukan secara berulang hingga seluruh data menjadi terurut. Algoritma selection sort memiliki konsep sederhana karena hanya melakukan proses pencarian nilai minimum atau maksimum lalu menempatkannya pada posisi yang benar.

Selain selection sort, terdapat juga algoritma insertion sort. Insertion sort merupakan algoritma pengurutan yang bekerja dengan cara menyisipkan suatu elemen ke posisi yang sesuai pada bagian array yang sudah terurut. Pada setiap proses perulangan, data akan dibandingkan dengan elemen sebelumnya, kemudian digeser sampai ditemukan posisi yang tepat. Algoritma ini cocok digunakan untuk data berukuran kecil karena proses pengurutannya cukup sederhana dan mudah dipahami.

Dalam implementasinya menggunakan bahasa Go, algoritma sorting biasanya diterapkan pada array maupun struct. Array digunakan untuk menyimpan sekumpulan data dengan tipe yang sama, sedangkan struct digunakan untuk menyimpan data dengan beberapa atribut berbeda dalam satu kesatuan, seperti data buku yang terdiri dari judul, penulis, penerbit, tahun, dan rating. Penggunaan struct membuat program lebih terstruktur dan mudah dikelola.

Pada modul ini, algoritma selection sort dan insertion sort digunakan untuk menyelesaikan berbagai studi kasus, seperti pengurutan nomor rumah kerabat, pencarian median data, pengecekan jarak data, dan pengelolaan data buku perpustakaan. Dengan mempelajari algoritma sorting, praktikan dapat memahami konsep dasar pengurutan data serta penerapannya dalam penyelesaian masalah menggunakan bahasa pemrograman Go.

## CONTOH SOAL

### 1. Latihan1

Source Code:

Output:

Deskripsi Program:

## SOAL LATIHAN SELECTION

### Statement

#### 1. rumahkerabat

### Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort( $T$  *arrInt,  $n$  int) {
    var i, j, idxMin, temp int

    i = 1

    for i <= n-1 {

        idxMin = i - 1
        j = i

        for j < n {

            if T[idxMin] > T[j] {
                idxMin = j
            }

            j = j + 1
        }

        temp = T[idxMin]
        T[idxMin] = T[i-1]
```

```

        T[i-1] = temp

        i = i + 1
    }
}

func main() {
    var daerah int
    var m, i, j int
    var rumah arrInt

    fmt.Print("Masukkan banyak daerah kerabat: ")
    fmt.Scan(&daerah)

    for i = 0; i < daerah; i++ {

        fmt.Println("\nDaerah ke-", i+1)

        fmt.Print("Masukkan banyak rumah kerabat: ")
        fmt.Scan(&m)

        fmt.Println("Masukkan nomor rumah kerabat:")

        for j = 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(&rumah, m)

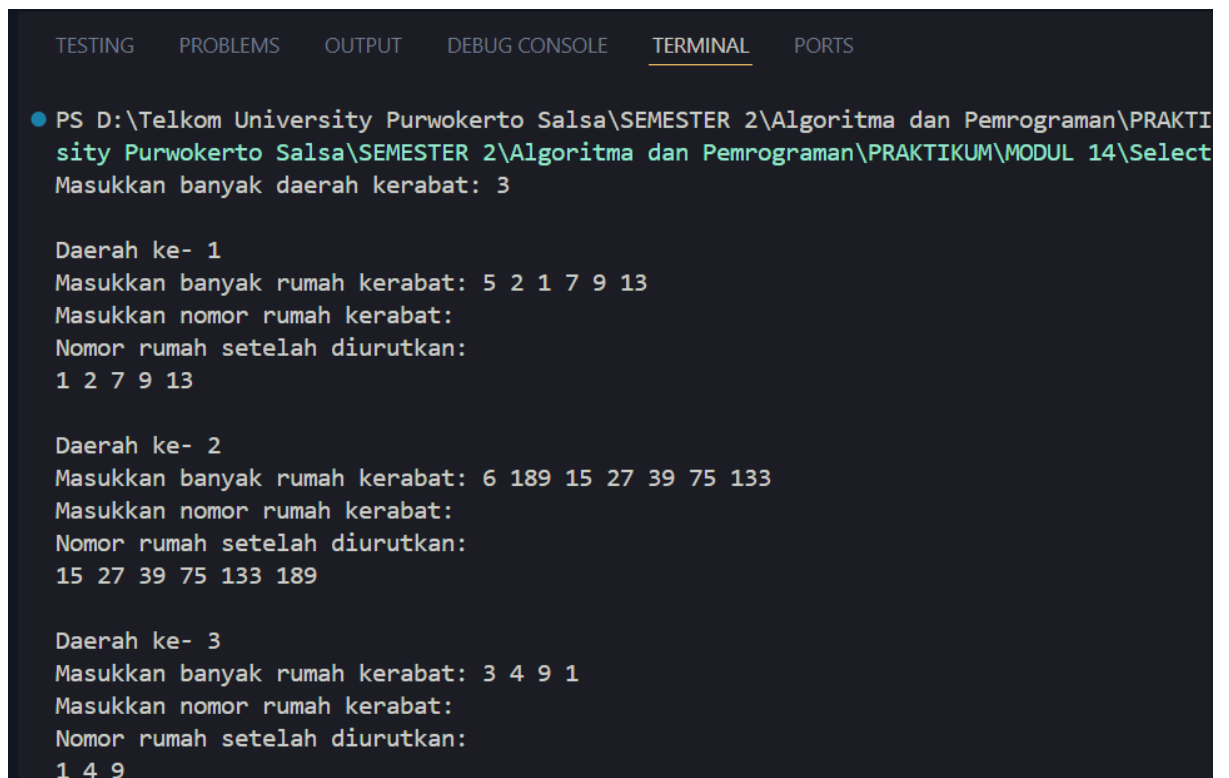
        fmt.Println("Nomor rumah setelah diurutkan:")

        for j = 0; j < m; j++ {
            fmt.Print(rumah[j], " ")
        }
    }
}

```

```
        fmt.Println()
    }
}
```

Output:



```
TESTING  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTI
sity Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTIKUM\MODUL 14>Select
Masukkan banyak daerah kerabat: 3

Daerah ke- 1
Masukkan banyak rumah kerabat: 5 2 1 7 9 13
Masukkan nomor rumah kerabat:
Nomor rumah setelah diurutkan:
1 2 7 9 13

Daerah ke- 2
Masukkan banyak rumah kerabat: 6 189 15 27 39 75 133
Masukkan nomor rumah kerabat:
Nomor rumah setelah diurutkan:
15 27 39 75 133 189

Daerah ke- 3
Masukkan banyak rumah kerabat: 3 4 9 1
Masukkan nomor rumah kerabat:
Nomor rumah setelah diurutkan:
1 4 9
```

Deskripsi Program:

Program di atas digunakan untuk mengurutkan nomor rumah kerabat Hercules di beberapa daerah menggunakan algoritma selection sort secara ascending atau membesar. Program menerima input berupa jumlah daerah yang akan diproses. Setiap daerah memiliki sejumlah nomor rumah kerabat yang disimpan ke dalam array. Setelah seluruh nomor rumah pada suatu daerah dibaca, program akan melakukan proses pengurutan menggunakan algoritma selection sort. Pada algoritma ini, program akan mencari nilai terkecil dari data yang belum terurut, kemudian menukarnya dengan elemen pada posisi paling depan dari bagian data yang belum terurut tersebut. Proses ini dilakukan berulang sampai seluruh data menjadi terurut. Setelah proses pengurutan

selesai dilakukan, program akan menampilkan nomor rumah kerabat Hercules dalam urutan membesar pada setiap daerah sesuai dengan data yang telah dimasukkan pengguna.

## 2. kerabatdekat

### Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort(T *arrInt, n int) {
    var i, j, idxMin, temp int

    i = 1

    for i <= n-1 {

        idxMin = i - 1
        j = i

        for j < n {

            if T[idxMin] > T[j] {
                idxMin = j
            }

            j = j + 1
        }
    }
}
```

```

        temp = T[idxMin]
        T[idxMin] = T[i-1]
        T[i-1] = temp

        i = i + 1
    }
}

func main() {
    var daerah int
    var m, i, j int
    var angka arrInt

    fmt.Print("Masukkan jumlah daerah: ")
    fmt.Scan(&daerah)

    for i = 0; i < daerah; i++ {

        fmt.Println("\nDaerah ke-", i+1)

        fmt.Print("Masukkan jumlah nomor rumah: ")
        fmt.Scan(&m)

        fmt.Println("Masukkan nomor rumah kerabat:")

        for j = 0; j < m; j++ {
            fmt.Scan(&angka[j])
        }

        selectionSort(&angka, m)

        fmt.Println("Urutan rumah kerabat:")

        for j = 0; j < m; j++ {

```



```

        if angka[j]%2 == 1 {
            fmt.Print(angka[j], " ")
        }
    }

    for j = m - 1; j >= 0; j-- {
        if angka[j]%2 == 0 {
            fmt.Print(angka[j], " ")
        }
    }

    fmt.Println()
}
}

```

Output:

```

TESTING  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTIKUM\dekat.go"
Masukkan jumlah daerah: 3

Daerah ke- 1
Masukkan jumlah nomor rumah: 5 2 1 7 9 13
Masukkan nomor rumah kerabat:
Urutan rumah kerabat:
1 7 9 13 2

Daerah ke- 2
Masukkan jumlah nomor rumah: 6 189 15 27 39 75 133
Masukkan nomor rumah kerabat:
Urutan rumah kerabat:
15 27 39 75 133 189

Daerah ke- 3
Masukkan jumlah nomor rumah: 3 4 9 1
Masukkan nomor rumah kerabat:
Urutan rumah kerabat:
1 9 4
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTIKUM\dekat.go"

```

#### Deskripsi Program:

Program di atas digunakan untuk mengurutkan nomor rumah kerabat Hercules berdasarkan aturan tertentu, yaitu nomor rumah ganjil ditampilkan lebih dahulu secara ascending atau membesar, kemudian diikuti nomor rumah genap secara descending atau mengecil. Program menerima input berupa jumlah daerah beserta nomor rumah kerabat pada masing-masing daerah. Seluruh nomor rumah terlebih dahulu disimpan ke dalam array, kemudian diurutkan menggunakan algoritma selection sort secara ascending. Pada algoritma selection sort, program mencari nilai terkecil dari data yang belum terurut lalu menukarnya dengan elemen pada posisi awal bagian data tersebut hingga seluruh array menjadi terurut. Setelah proses pengurutan selesai dilakukan, program akan mencetak seluruh bilangan ganjil terlebih dahulu dari kiri ke kanan sehingga urut membesar. Setelah itu program mencetak seluruh bilangan genap dari kanan ke kiri sehingga urut mengecil. Dengan demikian, keluaran program sesuai dengan aturan yang diberikan pada soal.

### 3. Median

#### Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort(T *arrInt, n int) {
    var i, j, idxMin, temp int

    i = 1

    for i <= n-1 {
```

```

        idxMin = i - 1
        j = i

        for j < n {

            if T[idxMin] > T[j] {
                idxMin = j
            }

            j = j + 1
        }

        temp = T[idxMin]
        T[idxMin] = T[i-1]
        T[i-1] = temp

        i = i + 1
    }
}

func median(T arrInt, n int) int {

    if n%2 == 1 {
        return T[n/2]
    }

    return (T[(n/2)-1] + T[n/2]) / 2
}

func main() {
    var data arrInt
    var x, n int

    fmt.Println("Masukkan data bilangan:")

```

```

    fmt.Println("(Masukkan 0 untuk mencari median)")
    fmt.Println("(Masukkan -5313 untuk mengakhiri program)")

    fmt.Scan(&x)

    for x != -5313 {

        if x == 0 {

            selectionSort(&data, n)

            fmt.Println("Median data:", median(data, n))

        } else {

            data[n] = x
            n++

        }

        fmt.Scan(&x)
    }
}

```

Output:

```

● PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algor
ity Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograma
Masukkan data bilangan:
(Masukkan 0 untuk mencari median)
(Masukkan -5313 untuk mengakhiri program)
7 23 11 0 5 19 2 29 3 13 17 0 -5313
Median data: 11
Median data: 12
○ PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algor

```

#### Deskripsi Program:

Program di atas digunakan untuk menentukan nilai median dari sekumpulan data bilangan bulat positif yang dimasukkan pengguna. Program menerima input berupa rangkaian bilangan bulat. Setiap bilangan selain 0 dan -5313 akan disimpan ke dalam array sebagai data yang akan diolah. Bilangan 0 digunakan sebagai tanda bahwa program harus menghitung dan menampilkan median dari seluruh data yang telah tersimpan saat itu. Sedangkan bilangan -5313 digunakan sebagai penanda akhir program sehingga proses input dihentikan. Sebelum median dihitung, seluruh data diurutkan terlebih dahulu menggunakan algoritma selection sort secara ascending atau membesar. Pada algoritma ini, program mencari nilai terkecil dari bagian array yang belum terurut lalu menukarnya dengan elemen di posisi depan sampai seluruh data menjadi terurut. Setelah data berhasil diurutkan, program akan menentukan median berdasarkan jumlah data yang tersimpan. Jika jumlah data ganjil, median diambil dari elemen tengah array. Namun jika jumlah data genap, median diperoleh dari rata-rata dua elemen tengah dan hasilnya dibulatkan ke bawah karena seluruh data berupa bilangan bulat. Setiap kali pengguna memasukkan angka 0, program akan menampilkan median terbaru dari data yang telah tersimpan sebelumnya.

## SOAL LATIHAN INSERTION

### Statement

#### 1. Data Berjarak Tetap

### Source Code:

```
package main

import "fmt"

type arrInt [1000]int

func insertionSort(T *arrInt, n int) {
    var i, j, temp int
```

```

i = 1

for i <= n-1 {

    j = i
    temp = T[j]

    for j > 0 && temp < T[j-1] {

        T[j] = T[j-1]
        j = j - 1
    }

    T[j] = temp
    i = i + 1
}

func main() {
    var data arrInt
    var n, x, i int
    var selisih int
    var tetap bool = true

    fmt.Println("Masukkan bilangan bulat:")
    fmt.Println("(Masukkan bilangan negatif untuk berhenti)")

    fmt.Scan(&x)

    for x >= 0 {

        data[n] = x
        n++
    }
}

```

```
        fmt.Scan(&x)
    }

    insertionSort(&data, n)

    fmt.Println("Data setelah diurutkan:")

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()

    selisih = data[1] - data[0]

    for i = 2; i < n; i++ {

        if data[i]-data[i-1] != selisih {
            tetap = false
        }
    }

    if tetap {
        fmt.Println("Data berjarak", selisih)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
```

Output:

```
TESTING PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\
  tetap.go"
  Masukkan bilangan bulat:
  (Masukkan bilangan negatif untuk berhenti)
  31 13 25 43 1 7 19 37 -5
  Data setelah diurutkan:
  1 7 13 19 25 31 37 43
  Data berjarak 6
○ PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\
  tetap.go"
```

### Deskripsi Program:

Program di atas digunakan untuk membaca sekumpulan bilangan bulat dari pengguna yang diakhiri dengan bilangan negatif. Bilangan negatif tidak disimpan ke dalam array dan hanya digunakan sebagai tanda akhir input. Semua bilangan non-negatif yang dimasukkan pengguna akan disimpan ke dalam array untuk kemudian diurutkan menggunakan algoritma insertion sort secara ascending atau membesar. Pada proses insertion sort, program akan menyisipkan setiap elemen ke posisi yang sesuai sehingga data menjadi terurut dari nilai terkecil ke terbesar. Setelah proses pengurutan selesai dilakukan, program akan menampilkan seluruh isi array yang telah terurut. Selanjutnya program akan memeriksa apakah setiap data di dalam array memiliki selisih atau jarak yang tetap antar elemennya. Program menghitung selisih dua elemen pertama sebagai acuan, kemudian membandingkannya dengan selisih elemen-elemen berikutnya. Jika semua selisih bernilai sama, maka program akan menampilkan tulisan "Data berjarak x", dengan x adalah nilai selisih tersebut. Namun apabila terdapat selisih yang berbeda, maka program akan menampilkan tulisan "Data berjarak tidak tetap".

## 2. Perpustakaan

### Source Code:

```
package main
```



```
import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    var i int

    for i = 0; i < n; i++ {

        fmt.Println("\nData Buku ke-", i+1)

        fmt.Print("Masukkan ID Buku      : ")
        fmt.Scan(&pustaka[i].id)

        fmt.Print("Masukkan Judul Buku    : ")
        fmt.Scan(&pustaka[i].judul)

        fmt.Print("Masukkan Penulis Buku  : ")
        fmt.Scan(&pustaka[i].penulis)

        fmt.Print("Masukkan Penerbit Buku : ")
        fmt.Scan(&pustaka[i].penerbit)

        fmt.Print("Masukkan Jumlah Buku   : ")
        fmt.Scan(&pustaka[i].eksemplar)

        fmt.Print("Masukkan Tahun Terbit  : ")
```

```

        fmt.Scan(&pustaka[i].tahun)

        fmt.Print("Masukkan Rating Buku  : ")
        fmt.Scan(&pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var i, idx int

    idx = 0

    for i = 1; i < n; i++ {

        if pustaka[i].rating > pustaka[idx].rating {
            idx = i
        }
    }

    fmt.Println("\nData Buku Terfavorit")
    fmt.Println("Judul    :", pustaka[idx].judul)
    fmt.Println("Penulis  :", pustaka[idx].penulis)
    fmt.Println("Penerbit :", pustaka[idx].penerbit)
    fmt.Println("Tahun    :", pustaka[idx].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var i, j int
    var temp Buku

    i = 1

    for i <= n-1 {

```

```

    j = i
    temp = pustaka[j]

    for j > 0 && temp.rating > pustaka[j-1].rating {

        pustaka[j] = pustaka[j-1]
        j = j - 1
    }

    pustaka[j] = temp
    i = i + 1
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var i int

    fmt.Println("\n5 Buku Dengan Rating Tertinggi")

    if n > 5 {
        n = 5
    }

    for i = 0; i < n; i++ {
        fmt.Println(i+1, ".", pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    var kiri, kanan, tengah int
    var found bool = false

    kiri = 0
    kanan = n - 1

```

```
for kiri <= kanan && !found {

    tengah = (kiri + kanan) / 2

    if pustaka[tengah].rating == r {

        found = true

    } else if r < pustaka[tengah].rating {

        kiri = tengah + 1

    } else {

        kanan = tengah - 1

    }

}

if found {

    fmt.Println("\nData Buku Ditemukan")
    fmt.Println("Judul      :", pustaka[tengah].judul)
    fmt.Println("Penulis   :", pustaka[tengah].penulis)
    fmt.Println("Penerbit  :", pustaka[tengah].penerbit)
    fmt.Println("Tahun     :", pustaka[tengah].tahun)
    fmt.Println("Eksemplar :", pustaka[tengah].eksemplar)
    fmt.Println("Rating    :", pustaka[tengah].rating)

} else {

    fmt.Println("\nTidak ada buku dengan rating seperti itu")

}

}
```

```
func main() {  
    var pustaka DaftarBuku  
    var n, ratingCari int  
  
    fmt.Print("Masukkan jumlah buku: ")  
    fmt.Scan(&n)  
  
    DaftarkanBuku(&pustaka, n)  
  
    CetakTerfavorit(pustaka, n)  
  
    UrutBuku(&pustaka, n)  
  
    Cetak5Terbaru(pustaka, n)  
  
    fmt.Print("\nMasukkan rating buku yang dicari: ")  
    fmt.Scan(&ratingCari)  
  
    CariBuku(pustaka, n, ratingCari)  
}
```

Output:

```

PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTIKUM\MODUL 14> go run
sertion\program perpustakaan\2.programperpustakaan.go"
Masukkan jumlah buku: 2

Data Buku ke- 1
Masukkan ID Buku      : B01
Masukkan Judul Buku   : Naruto
Masukkan Penulis Buku : Kishimoto
Masukkan Penerbit Buku : Shueisha
Masukkan Jumlah Buku  : 10
Masukkan Tahun Terbit : 2005
Masukkan Rating Buku  : 90

Data Buku ke- 2
Masukkan ID Buku      : B02
Masukkan Judul Buku   : OnePiece
Masukkan Penulis Buku : Oda
Masukkan Penerbit Buku : Jump
Masukkan Jumlah Buku  : 8
Masukkan Tahun Terbit : 2001
Masukkan Rating Buku  : 95

Data Buku Terfavorit
Judul      : OnePiece
Penulis    : Oda
Penerbit   : Jump
Tahun      : 2001

5 Buku Dengan Rating Tertinggi
1 . OnePiece
2 . Naruto

Masukkan rating buku yang dicari: 90

Data Buku Ditemukan
Judul      : Naruto
Penulis    : Kishimoto
Penerbit   : Shueisha
Tahun      : 2005
Eksemplar  : 10
Rating     : 90
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pemrograman\PRAKTIKUM\MODUL 14>

```

### Deskripsi Program:

Program di atas digunakan untuk mengelola data buku pada sebuah perpustakaan. Program menerima sejumlah data buku yang terdiri dari ID buku, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating buku. Semua data tersebut disimpan ke dalam array bertipe struct agar data buku dapat dikelola dengan lebih terstruktur. Program memiliki beberapa subprogram untuk membantu proses pengolahan data. Subprogram DaftarkanBuku digunakan untuk membaca dan menyimpan seluruh data buku yang dimasukkan pengguna ke dalam array pustaka. Setelah seluruh data berhasil disimpan, program akan mencari buku dengan rating tertinggi menggunakan subprogram CetakTerfavorit. Program kemudian menampilkan informasi buku terfavorit berupa judul, penulis, penerbit, dan tahun terbit. Selanjutnya program menggunakan subprogram UrutBuku untuk mengurutkan data buku berdasarkan rating secara descending atau menurun menggunakan algoritma insertion sort. Pada algoritma

insertion sort, setiap data buku akan disisipkan ke posisi yang sesuai sehingga seluruh data menjadi terurut berdasarkan rating tertinggi hingga terendah. Setelah proses pengurutan selesai dilakukan, program akan menampilkan lima buku dengan rating tertinggi menggunakan subprogram Cetak5Terbaru. Jika jumlah data buku kurang dari lima, maka program hanya akan menampilkan seluruh data yang tersedia.

Program juga memiliki subprogram CariBuku yang digunakan untuk mencari buku berdasarkan rating tertentu menggunakan metode binary search atau pencarian biner. Jika buku dengan rating yang dicari ditemukan, maka program akan menampilkan informasi lengkap buku tersebut. Namun jika tidak ditemukan, program akan menampilkan pesan bahwa tidak ada buku dengan rating seperti itu.

## DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2 – Modul 14 Sorting. Fakultas Informatika, Telkom University Purwokerto.

Noval Agung. Dasar Pemrograman Golang.

<https://dasarpemrogramangolang.novalagung.com>

Kadir, Abdul. 2018. *Dasar Algoritma dan Pemrograman Menggunakan Go*. Yogyakarta: Andi Offset.